

Tensor Decompositions and Applications - T.G. Kolda, B.W. Bader

Inner product of two same-sized tensors $X, Y \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_r}$

$$\langle X, Y \rangle = \sum_{i_1=1}^{I_1} \sum_{i_2=1}^{I_2} \dots \sum_{i_r=1}^{I_r} x_{i_1, i_2, \dots, i_r} y_{i_1, i_2, \dots, i_r}$$

```
{ def Inn_Pro ( X, Y ) :  
    I1 = len ( X )  
    :  
    Ir = len ( X[0][0]...[0] )  
    Sum = 0.0  
    for i1 in range ( I1 ) :  
        for i2 in range ( I2 ) :  
            :  
            :  
            for ir in range ( Ir :  
                Sum += X[i1, i2, ..., ir] * Y[i1, i2, ..., ir]  
    return Sum
```

- Frobenius norm -

The norm of a tensor $X \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_r}$ is the square root of the sum of the squares of all its elements

$$\|X\| = \sqrt{\sum_{i_1=1}^{I_1} \sum_{i_2=1}^{I_2} \dots \sum_{i_r=1}^{I_r} x_{i_1, i_2, \dots, i_r}^2}$$

```
{ def Fro_Norm ( X )  
    return np.sqrt ( Inn_Pro ( X, X ) )
```

- CP-Decomposition -

The CP decomposition factorizes a tensor into a sum of component rank-one tensors. For example, given a third-order tensor $X \in \mathbb{R}^{I \times J \times K}$

$$X \approx \sum_{r=1}^R a_r \circ b_r \circ c_r$$

(where, R is a positive integer, $a_r \in \mathbb{R}^I$, $b_r \in \mathbb{R}^J$, $c_r \in \mathbb{R}^K$)
Elementwise, It is written as

$$x_{ijk} = \sum_{r=1}^R a_{ir} b_{jr} c_{kr} \quad (for \ i=1, \dots, I, \ j=1, \dots, J, \ k=1, \dots, K)$$

- Khatri-Rao product -

$$A \odot B = [a_1 \otimes b_1 \quad a_2 \otimes b_2 \quad \dots \quad a_R \otimes b_R]$$

(where $A \in \mathbb{R}^{I \times R}$, $B \in \mathbb{R}^{J \times R}$)

matrices column $\gg$ $\frac{I}{J}$, $A \odot B \in \mathbb{R}^{(IJ \times R)}$

- The factor matrices refer to the combination of the vectors from the rank-one components ($A = [a_1 \ a_2 \ \dots \ a_R]$, likewise for B and C). Using CP-decomposition's definitions, it may be written in matricized form

$$X_{(1)} \approx A (C \odot B)^T$$

Size: $I \times R \cdot R \times (JK) \rightarrow I \times (JK)$

$$X_{(2)} \approx B (C \odot A)^T$$

$$X_{(3)} \approx C (B \odot A)^T$$

e.g.

$$\mathbb{X} \in \mathbb{R}^{3 \times 4 \times 2}$$

$$\mathbb{X}_1 = \begin{bmatrix} 1 & 4 & 7 & 10 \\ 2 & 5 & 8 & 11 \\ 3 & 6 & 9 & 12 \end{bmatrix}, \quad \mathbb{X}_2 = \begin{bmatrix} 13 & 16 & 19 & 22 \\ 14 & 17 & 20 & 23 \\ 15 & 18 & 21 & 24 \end{bmatrix}$$

CP decomposition $\rightarrow \mathbb{X}'$ 2-rank 1 tensor

$$\mathbb{X}' = a_1 \otimes b_1 \otimes c_1 + a_2 \otimes b_2 \otimes c_2$$

p factor matrices

$$\left(\begin{array}{l} A = [a_1 \ a_2] = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \\ B = [b_1 \ b_2] = \begin{bmatrix} 1 & 2 \\ 0 & 1 \\ 2 & 0 \\ 1 & 1 \end{bmatrix} \\ C = [c_1 \ c_2] = \begin{bmatrix} 1 & 1 \\ 0 & 2 \end{bmatrix} \end{array} \right)$$

$$C \odot B = [c_1 \otimes b_1 \quad c_2 \otimes b_2]$$

$$c_1 \otimes b_1 = \begin{bmatrix} 1 \times \begin{bmatrix} 1 \\ 0 \\ 2 \\ 1 \end{bmatrix} \\ 0 \times \begin{bmatrix} 1 \\ 0 \\ 2 \\ 1 \end{bmatrix} \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 2 \\ 1 \\ 0 \\ 0 \\ 2 \\ 1 \end{bmatrix}$$

$$c_2 \otimes b_2 = \begin{bmatrix} 1 \times \begin{bmatrix} 2 \\ 1 \\ 0 \\ 1 \end{bmatrix} \\ 2 \times \begin{bmatrix} 2 \\ 1 \\ 0 \\ 1 \end{bmatrix} \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \\ 0 \\ 1 \\ 4 \\ 2 \\ 0 \\ 2 \end{bmatrix}$$

$$\Rightarrow C \odot B = \begin{bmatrix} 1 & 2 \\ 0 & 1 \\ 2 & 0 \\ 1 & 1 \\ 0 & 4 \\ 0 & 2 \\ 0 & 0 \\ 0 & 2 \end{bmatrix}$$

$$X_{(1)} = A(C \odot B)^T = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 2 & 1 & 0 & 0 & 0 & 0 \\ 2 & 1 & 0 & 1 & 4 & 2 & 0 & 2 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 0 & 2 & 1 & 0 & 0 & 0 & 0 \\ 2 & 1 & 0 & 1 & 4 & 2 & 0 & 2 \\ 3 & 1 & 2 & 2 & 4 & 2 & 0 & 2 \end{bmatrix}$$

- Metricization (unfolding or flattening) -

The mode- n matricization of a tensor $\mathbb{X} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_N}$ is denoted by $\mathbb{X}_{(n)}$ and arranges the mode- n fibers to be the columns of the resulting matrix.

$$\text{Tensor } \mathbb{R}^{I_1 \times I_2 \times \dots \times I_N} \rightarrow \text{Matrix } \mathbb{R}^{I_n \times (I_1 \times \dots \times I_{n-1} \times I_{n+1} \times \dots \times I_N)}$$

$$(i_1, i_2, \dots, i_N) \mapsto (i_n, 1 + \sum_{\substack{k=1 \\ k \neq n}}^N (i_k - 1) J_k)$$

(where $J_k = \prod_{\substack{m=1 \\ m \neq k}}^{k-1} I_m$)

$m \neq n$

↑

열 종이의 자리 수 개수 21

(e.g.) $X \in \mathbb{R}^{3 \times 4 \times 2}$

$$X_1 = \begin{bmatrix} 1 & 4 & 7 & 10 \\ 2 & 5 & 8 & 11 \\ 3 & 6 & 9 & 12 \end{bmatrix}, \quad X_2 = \begin{bmatrix} 13 & 16 & 19 & 22 \\ 14 & 17 & 20 & 23 \\ 15 & 18 & 21 & 24 \end{bmatrix}$$

$$\cancel{X}_{(1)} = \begin{bmatrix} 1 & 4 & 7 & 10 & 13 & 16 & 19 & 22 \\ 2 & 5 & 8 & 11 & 14 & 17 & 20 & 23 \\ 3 & 6 & 9 & 12 & 15 & 18 & 21 & 24 \end{bmatrix}$$

$$x_{2,3,2} = x_{(1)2,7}$$

Since, $j = 1 + (i_2 - 1)J_2 + (i_3 - 1)J_3$
 $= 1 + (3-1) \times 1 + (2-1) \times 4$
 $= 1 + 2 + 4 = 7$

```
def Find_Flattenig_index ( Origin_Ten , Origin_ind , mode )
    mode -= 1
    Flat_ind = [ Origin_ind [mode] , 0 ]
    I = [ ]
```

```

dummy = Origin-Ten.copy()
for _ in range(len(origin-ind)):
    I.append(len(dummy))
    dummy = dummy[0]

```

j = 1

```

for k in range(len(origin-ind)):
    if (k == mode):
        pass

```

J = 1

```

for m in range(k):
    if (m == k) or (m == mode):
        pass

```

J *= I[m]

j += J * (Origin-ind[k] - 1)

Flat-ind[1] = j

return Flat-ind

- Alternating Least Square (ALS) method -

Let $X \in \mathbb{R}^{I \times J \times K}$ be a third-order tensor. The goal is to compute a CP decomposition with R components that best approximates X , i.e., to find

$$\min_{X'} \|X - X'\| \quad \text{with} \quad X' = \sum_{r=1}^R \lambda_r a_r \circ b_r \circ c_r$$

$$= [\lambda; A, B, C]$$

(where $A \in \mathbb{R}^{I \times R}$, $B \in \mathbb{R}^{J \times R}$, $C \in \mathbb{R}^{K \times R}$)

Having fixed all but one matrix, the problem reduces to a linear least-squares problem. For example, suppose that B and C are fixed. Then, we can rewrite the above minimization problem in matrix form as

$$\min_{A'} \|X_{(1)} - A'(C \odot B)^T\|_F,$$

where $A' = A \cdot \text{diag}(\lambda)$. The optimal solution is then given by

$$X_{(1)} - A'(C \odot B)^T \approx 0$$

$$\Rightarrow A' = X_{(1)} \underbrace{[(C \odot B)^T]^+}_{\mathbb{R}^{(JK) \times R}} = X_{(1)} [(C \odot B)^+]^T$$

$$\text{Since, } M^+ = (M^T M)^+ M^T$$

$$\Rightarrow (C \odot B)^+ = [(C \odot B)^T (C \odot B)]^+ (C \odot B)^T$$

$$\text{Since, } \underbrace{[(C \odot B)^T (C \odot B)]^+}_{\mathbb{R}^{R \times R}} = \underbrace{C^T C}_{\mathbb{R}^{JK \times R}} \underbrace{* B^T B}_{\mathbb{R}^{R \times R}} \quad \text{Hadamard product}$$

$$\therefore A' = X_{(1)} \underbrace{(C \odot B)}_{\mathbb{R}^{(JK) \times R}} \underbrace{(C^T C * B^T B)^+}_{\mathbb{R}^{R \times R}}$$

dagger에 들어가는 행렬의 사이즈 맞추기.

Finally, we normalize the columns of A' to get A , in other words, let $\lambda_v = \|a'_v\|$ and $a_v = a'_v / \lambda_v$

for $r = 1, \dots, R$

The factor matrices can be initialized in any way,
such as randomly or by setting

$\mathbf{A}^{(n)} = \mathbf{R}$ leading left singular vectors of $\mathbf{X}^{(n)}$ for $n=1, \dots, N$
 $= \mathbf{A}, \mathbf{B}, \mathbf{C}$

```

procedure CP-ALS( $\mathbf{X}, R$ )
  initialize  $\mathbf{A}^{(n)} \in \mathbb{R}^{I_n \times R}$  for  $n=1, \dots, N$ 
  repeat
    for  $n=1, \dots, N$  do
       $\mathbf{V} \leftarrow \mathbf{A}^{(1)\top} \mathbf{A}^{(1)} * \dots * \mathbf{A}^{(n-1)\top} \mathbf{A}^{(n-1)} * \mathbf{A}^{(n+1)\top} \mathbf{A}^{(n+1)} * \dots * \mathbf{A}^{(N)\top} \mathbf{A}^{(N)}$ 
       $\mathbf{A}^{(n)} \leftarrow \mathbf{X}^{(n)} (\mathbf{A}^{(N)} \odot \dots \odot \mathbf{A}^{(n+1)} \odot \mathbf{A}^{(n-1)} \odot \dots \odot \mathbf{A}^{(1)}) \mathbf{V}^\dagger$ 
      normalize columns of  $\mathbf{A}^{(n)}$  (storing norms as  $\lambda$ )
    end for
  until fit ceases to improve or maximum iterations exhausted
  return  $\lambda, \mathbf{A}^{(1)}, \mathbf{A}^{(2)}, \dots, \mathbf{A}^{(N)}$ 
end procedure
  
```

Fig. 3.3 ALS algorithm to compute a CP decomposition with R components for an N th-order tensor $\mathbf{X}$ of size $I_1 \times I_2 \times \dots \times I_N$.

for ~
 {
 if (Norm < δ):
 break
 else
 print ("issue")

~ Other CP-algorithms ~

(ASD) : flattening 연산이 3차원 텐서로 수행. ALS에 비해 정확도가 높을 수 있음.

(dGN) Damped Gauss-Newton method : 수렴 속도가 빠름.

(PMF3) : dGN을 실용적으로 개발, 노이즈가 있는 실제 데이터에 적용.
 성분 고정도 안시키고 한 번에 최적화.

Solution에 가까워질수록 수렴 속도가 기하급수적으로 빨라짐.

높은 메모리 및 연산 비용 소모.

- dGN based PMF3-

로그 패리티 함수로 원소들이 Non-negative 하게
만드는게 PMF3의 핵심인데 복소수 공간에서는 제거해야 함.

오버 스텝: PMF3는 대각로 행렬 연산들 위해 무거운
Hessian 구조 알고리즘 이용.

$\mathbb{C}^3 \otimes \mathbb{C}^3 \otimes \mathbb{C}^3$ 에서 연산 시간은 얼마 안 걸림.